Architecting a Stateless Baseball Franchise Simulation Engine: A Mathematical and Sabermetric Blueprint

The development of a browser-based baseball franchise dynasty simulation game, executed entirely within a client-side framework such as React and JSX, presents unique computational and architectural challenges. Operating without a backend server requires the engine to function within the strict memory and processing constraints of the user's browser. To achieve deep statistical credibility while maintaining rapid, synchronous execution loops, the simulation cannot rely on computationally expensive Monte Carlo physical physics engines. Instead, the architecture must be grounded in deterministic mathematical formulas and rigorous stochastic models that instantly map player ratings to real-world statistical outcomes.

The core objective of this intelligence report is to provide a comprehensive, actionable blueprint of the actual numbers, formulas, and sabermetric frameworks required to build a statistically credible simulation engine. By integrating advanced probability mathematics—ranging from multi-outcome Log5 matchup calculations and Markov chain run expectancy models to dynamic Pythagenpat standings and biomechanical fatigue curves—the engine can simulate individual pitches, in-game managerial decisions, and multi-decade career trajectories with absolute fidelity. The following sections exhaustively detail the mechanics necessary to power a lightweight, fully self-contained baseball universe.

Generative Matchup Mathematics: The Log5 and Odds Ratio Framework

At the atomic level of any baseball simulation engine is the batter-pitcher matchup. For every plate appearance, the engine must determine the probability of a specific outcome (such as a strikeout, walk, single, double, or home run) based on the individual skill profiles of the batter and the pitcher, contextualized by the baseline average of the entire simulated league. The foundational approach for this calculation is the Log5 method.

The Standard Log5 and Binary Odds Ratio Mechanics

Originally conceptualized by statistician Bill James and Dallas Adams in the early 1980s, the standard Log5 method was initially designed to predict the probability of a binary outcome, such as whether a specific team wins a game or whether a specific batter registers a hit against a specific pitcher.[1] The basic binary Log5 formula operates on an odds ratio framework. If P_{batter} represents the batter's probability of achieving a specific event, $P_{pitcher}$ represents the pitcher's probability of allowing that event, and P_{league} represents the league average

probability for that event, the odds ratio (OR) and the inverse odds ratio (iOR) are calculated as follows [4]:

$$OR = \frac{P_{pitcher} \times P_{batter}}{P_{league}}$$

$$iOR = \frac{(1 - P_{pitcher}) \times (1 - P_{batter})}{(1 - P_{league})}$$

The final probability of the event occurring in this specific, isolated matchup is defined mathematically as [4]:

$$Probability = \frac{OR}{OR + iOR}$$

While this binary model is highly effective and computationally inexpensive for a JavaScript simulation loop, it possesses known mathematical failure modes. Academic research indicates that the standard Log5 formula contains a slight bias when applied to estimating a team or player's talent against league-average competition.[5] Because the formula inherently relies on a player's expectation against a strict 0.500 (average) baseline rather than their expectation averaged out across all possible varying opponents in a league distribution, it can be slightly too conservative, overestimating the underdog's chances.[5] However, within the normal range of Major League Baseball talent distributions, this discrepancy is mathematically negligible, often amounting to a deviation of merely one point of winning percentage.[5] For the purposes of a client-side simulation engine, the computational speed of the odds ratio far outweighs this minor bias.

The Multi-Outcome Log5 Generalization

A modern simulation engine cannot rely on binary outcomes; resolving a plate appearance requires partitioning the sample space into multiple, mutually exclusive events (e.g., Single, Double, Triple, Home Run, Walk, Hit-by-Pitch, Strikeout, and In-Play Out).[2] Extending the Log5 method to accommodate multiple outcomes requires the engine to calculate a "Base Probability" for each distinct event type and then normalize the results so that the sum of all probabilities exactly equals 1.0.[2]

Let x_i be the batter's true-talent seasonal probability for event i, y_i be the pitcher's true-talent seasonal probability for event i, and z_i be the league average probability for event i. The Base Probability for event i is defined as [2]:

$$\text{Base Probability}_i = \frac{x_i \cdot y_i}{z_i}$$

Because the sum of the base probabilities across all possible events in the partitioned sample space will not naturally sum to 1.0, the General Matchup Probability Formula is utilized to normalize the distribution [2]:

$$P(E_i) = \frac{\frac{x_i \cdot y_i}{z_i}}{\sum_{j=1}^{n} \left(\frac{x_j \cdot y_j}{z_j} \right)}$$

This normalization ensures that the sum of all $P(E_i)$ equals exactly 1.0, satisfies mathematical symmetry (the result is identical regardless of which player is defined as x or y), and guarantees that an exactly average player (z) facing a given player (x) will yield x's original probabilities.[2] In a single-file, browser-based execution loop, precomputing the denominator (the sum of all base probabilities) for a specific batter-pitcher matchup allows the engine to rapidly map a random number generation (RNG) float to a cumulative probability distribution array, instantly and deterministically resolving the plate appearance.

Enhancements and Failure Mode Corrections

While the generalized Log5 handles mathematical normalization perfectly, it suffers from two practical failure modes in simulation environments: small sample sizes and interaction variables. A statistically credible engine must account for both.

The generalized Log5 model allows coefficients to be estimated via logistic regression, but this breaks down entirely when players have insufficient historical data, such as newly generated rookie players or relief pitchers with exceptionally low innings counts.[6] To stabilize the simulation and prevent anomalous mathematical outputs (such as a 100% strikeout probability), the engine must implement a Bayesian hierarchical model.[6] By using the fixed coefficients of the standard Log5 model as prior knowledge and blending them with the player's observed data, the engine can estimate probabilities reliably even with minute amounts of historical data.[6] The simulation should maintain hidden "true talent" state variables for each player that serve as the Bayesian priors, heavily weighting the generation of Log5 odds until the player's simulated career provides an adequate sample size.

Furthermore, the standard Log5 assumes complete mathematical independence between variables, which is physically inaccurate for certain matchup types. Exhaustive research analyzing nearly one million real-world matchup observations reveals that a batter/pitcher ground ball rate interaction variable is highly significant for predicting strikeout probabilities and contact outcomes.[7] A pitcher who heavily induces ground balls mathematically neutralizes a batter whose profile relies on extreme fly-ball launch angles. To achieve realism, the simulation engine should apply a modifying coefficient to the Base Probability of an Out or an

Extra Base Hit based on the absolute difference between the pitcher's Ground Ball percentage and the batter's Fly Ball percentage.[7]

Platoon Splits and Asymmetrical Handedness Modifiers

Platoon splits—the variation in a batter's performance against opposite-handed versus same-handed pitching—are a critical component of lineup optimization and managerial AI logic. The simulation engine must implement these dynamically without mathematically distorting the underlying player evaluations.

The "Double-Counting" Strat-O-Matic Error

A critical architectural pitfall in simulation development is the "Strat-O-Matic error," named after the logic flaw present in early tabletop and digital baseball simulations. In these legacy systems, the simulation calculates a player's baseline rating from their total aggregated statistical output, and then applies a platoon modifier directly on top of it.[9] This mathematically "double-counts" the platoon split.[9]

If an average left-handed batter possesses a 25-point platoon split advantage in real life, applying a generic 25-point modifier to their aggregated overall stats in the simulation will result in a functional 50-point split in the game environment.[9] Because right-handed batters heavily outnumber left-handed batters in any realistic league generation, left-handed pitchers are severely penalized by this double-counting.[9] To circumvent this fatal mathematical flaw, the simulation engine must dynamically *de-normalize* the player's overall rating into neutral baseline components (Base vs. LHP and Base vs. RHP) during player generation, ensuring that the weighted sum of these split components perfectly equals the player's true total rating based on the exact ratio of left-handed to right-handed pitchers currently active in the simulated league.

Split Magnitudes and Stabilization

As a fundamental rule, hitters perform significantly better against opposite-handed pitchers.[10] However, the variation in platoon skill is not equal across handedness; left-handed batters display substantially wider variation in their platoon splits than right-handed batters.[11]

A baseline magnitude for platoon splits can be drawn from large-sample Major League Baseball data. The engine can use these baseline figures to generate hidden platoon modifiers for newly drafted players. Examining batting averages across a massive sample size reveals the following aggregate splits:

Matchup (Pitcher vs. Batter)	Average	Advantage Metric
RHP vs. RHB (Same Handed)	.251	Baseline
LHP vs. RHB (Opposite Handed)	.263	+12 points for Batter
RHP vs. LHB (Opposite Handed)	.260	+9 points for Batter
LHP vs. LHB (Same Handed)	.232	+28 points for Pitcher

(Data derived from large-sample split research [12])

The data clearly illustrates that left-handed pitchers possess a massive, disproportionate advantage when facing left-handed batters, suppressing batting average by nearly 30 points compared to the opposite-handed baseline.[12]

When generating or evaluating players with low sample sizes, the engine must regress their observed split toward the league mean to establish their true talent. Due to the wider variance inherent in left-handed profiles, a left-handed hitter's split should be regressed against 1,000 Plate Appearances versus LHP of league-average split data, whereas a right-handed hitter's split must be regressed against a much heavier 2,200 Plate Appearances versus LHP.[11] The engine's state management should slowly peel away this regression as the simulated player accrues real plate appearances within the game universe.

In-Game State Progression: Markov Chains and Run Expectancy

Once the multi-outcome Log5 model generates the atomic result of a plate appearance, the simulation engine must process the physical movement of runners on the bases and the accumulation of runs. The most rigorous and academically sound framework for modeling baseball progression without relying on complex, physics-based collision logic is the Markov

chain.[13]

The 25-State Matrix Architecture

The game of baseball operates within a discrete, rigidly defined state-based structure. A half-inning can be perfectly described by a finite set of states defined by the number of outs (0, 1, or 2) and the configuration of baserunners (Empty, 1st, 2nd, 3rd, 1st & 2nd, 1st & 3rd, 2nd & 3rd, and Bases Loaded).[13] This yields $3 \times 8 = 24$ active base-out states, plus an essential 25th "absorbing state" representing the end of the inning upon the recording of the third out.[13]

The engine must maintain a 24×24 transition matrix, denoted mathematically as P, which dictates the exact probability of transitioning from one state to another given the outcome generated by the Log5 logic.[13] For example, if the Log5 formula generates a single with a runner on first base and zero outs, the transition matrix dictates whether the state transitions to runners on first and second with zero outs, or runners on first and third with zero outs, depending on the baserunner's speed rating versus the outfielder's arm rating. The probabilities within this matrix are determined continuously by the simulated league's environment.[13]

Run Expectancy and the Fundamental Matrix

The true power of the Markov chain lies in its ability to dynamically calculate the Expected Runs (E) for the remainder of an inning from any given state. This calculation is vital not just for the simulation loop, but for the AI manager's decision-making logic, allowing the computer to determine the mathematical viability of a sacrifice bunt, a stolen base attempt, or an intentional walk.[13] The Run Expectancy equation is structured as:

$$E = (I - P)^{-1} R$$

Where:

- E represents the vector of run expectancies for each of the 24 base-out states.[13]
- I is a standard 24×24 Identity Matrix.[13]
- P is the 24×24 transition submatrix for the non-absorbing states.[13]
- $(I - P)^{-1}$ represents the fundamental matrix, which calculates the exact expected number of times the Markov chain will visit each state before being absorbed by the end of the inning.[13]
- R is the expected run vector, where each element R_i is the expected number of runs

scored during a single transition from state i.[13]

By solving this linear algebra equation during the initialization of the game environment, the engine maps a baseline Run Expectancy Matrix (RE24). For context, in modern run environments, the matrix maps as follows:

Base State	**0 Outs**	**1 Out**	**2 Outs**
Bases Empty	0.48	0.26	0.10
Runner on 1st	0.88	0.52	0.21
Runner on 2nd	1.10	0.69	0.32
Runner on 3rd	1.23	0.99	0.40
Runners on 1st & 2nd	1.48	0.93	0.44
Runners on 1st & 3rd	1.77	1.17	0.50
Runners on 2nd & 3rd	2.03	1.42	0.56
Bases Loaded	2.45	1.56	0.81

(Data reflecting expected runs per base-out state in recent historical MLB environments [20])

The engine uses this exact matrix to assign precise valuation to offensive events dynamically. For example, normalized over all 24 game states based on their natural frequency of occurrence, the expected run gain of a triple is 1.033 runs, whereas the expected run gain of a

home run is 1.420 runs.[17] This exposes a massive mathematical flaw in traditional Slugging Percentage (which falsely values a triple at 0.750 of a home run); the Markov probability model proves that a triple is only worth roughly 0.727 of a home run, meaning standard SLG overvalues the triple by approximately 3.08%.[17] The simulation engine's internal AI valuation systems must use these Markov-derived linear weights rather than traditional stat-line weights to evaluate trades, lineups, and contract values.

Managerial AI and Leverage Index (LI) Mathematics

To accurately simulate the in-game decision-making of an AI manager (e.g., when to deploy a closer, when to issue an intentional walk, or when to burn a premium pinch hitter), the engine requires a real-time, objective measurement of situational pressure. The sabermetric mathematical standard for this is the Leverage Index (LI), created by Tom Tango.[21]

Calculating Mid-Game Leverage Index

The Leverage Index measures the importance of a particular event by quantifying the exact extent to which the overall game Win Probability (WE) could change during a specific plate appearance.[21] An LI of 1.0 represents a perfectly neutral situation, an LI below 0.85 indicates a low-leverage situation, and an LI above 2.0 designates high leverage.[22] Statistically, only about 10% of game situations exceed an LI of 2.0.[22]

To calculate LI dynamically within the React engine loop, the simulation must measure the "swing" of potential changes in Win Expectancy [22]:

1. **Establish the Context:** Identify the specific inning, score differential, out count, and baserunner state to find the current baseline Win Expectancy (WE).[22]
2. **Determine Potential WE Deltas:** Calculate all possible new Win Expectancies resulting from every potential plate appearance outcome (Single, Home Run, Strikeout, Double Play, etc.), yielding a set of ΔWE values.[22]
3. **Apply Probability Weighting:** Multiply each potential ΔWE by the league-average probability of that outcome occurring (p_i).[22]
4. **Sum and Normalize:** Sum these probability-weighted swings and divide by the league-wide average potential swing in WE for all possible neutral situations. This neutral swing constant generally hovers around 0.04 depending on the run environment.[22]

$$LI = \frac{\sum(p_i \times |\Delta WE_i|)}{\text{Average Neutral WE Swing}}$$

Optimized Estimation for Rapid Simulation Loops

Running the full probabilistic derivation mid-game can be computationally expensive if

evaluating thousands of decisions per tick in a fast-forwarded simulation season. A mathematically viable "quick-and-dirty" estimation can be used by the engine's AI without sacrificing credibility.[25]

The engine can take the anticipated absolute Win Expectancy delta (ΔWE) of a standard run-scoring event in that situation, divide it by the linear weights run value of that event (e.g., 0.46 runs for a single), and divide the result by the typical win-to-run ratio constant, which is precisely 0.1.[25]

$$LI_{estimate} \approx \frac{\left(\frac{\Delta WE}{\text{Run Value}}\right)}{0.1}$$

The AI manager logic should monitor the LI threshold in real-time. If the $LI > 1.5$ in the late innings, the AI should trigger high-leverage bullpen routines.[23] If the $LI > 7.0$ (representing extreme leverage, such as bases loaded, two outs, down by one run in the bottom of the ninth), the AI may bypass traditional closer roles entirely and bring in their highest WAR reliever regardless of their assigned bullpen role.[23]

Biomechanical Fatigue and Velocity Degradation Models

Traditional simulation games often rely on arbitrary pitch count thresholds—such as Pitcher Abuse Points (PAP) triggering sharply after 100 pitches—to simulate fatigue and injury risk.[26] However, biomechanical and statistical research indicates this is entirely inaccurate. Pitchers do not degrade sharply at a magic number; fatigue is a continuous, cumulative biological process influenced heavily by pitch type, velocity, and medial elbow torque.[26]

Continuous Velocity Degradation

In a simulated game environment, the engine must model fatigue primarily through the continuous erosion of fastball velocity. Empirical tracking from simulated game studies demonstrates that average pitch velocity decreases by approximately 0.28 mph per successive inning pitched, while medial elbow torque concurrently increases.[28] Fastballs generate the highest medial elbow torque, meaning a pitcher relying heavily on four-seam fastballs will experience a steeper fatigue curve than a knuckleball or change-up specialist.[28]

Translating Velocity Loss to Run Prevention

As the pitcher's velocity degrades inning by inning within the simulation loop, the engine must translate this physical loss directly into the Log5 probability calculations.

The relationship between fastball velocity and batter effectiveness is stark. Batted-ball data

reveals that for every 1 mph of fastball velocity lost, there is a measurable increase in the batter's Expected Weighted On-Base Average (xwOBA).[30] Most critically, the transition through the upper velocity bands is non-linear. Pitchers throwing 98 mph and above suppress wOBA significantly. When velocity degrades from the 98+ mph bucket into the 96 to 98 mph bucket, the average allowed wOBA increases by a staggering 16%.[33]

The engine should structure its in-game fatigue loop as follows:

1. **Initialize Fatigue Index:** Set a baseline based on the pitcher's hidden endurance rating.
2. **Accumulate Load:** For every pitch thrown, increase the fatigue index. Multiply the base pitch cost by a stress factor (fastballs > breaking balls > off-speed) and a leverage factor. Pitches thrown in high-leverage ($LI > 1.5$) situations increase mental and neuromuscular fatigue significantly more than mop-up pitches.[28]
3. **Degrade Velocity:** Reduce the pitcher's effective velocity linearly by roughly 0.28 mph per equivalent inning of accumulated load.[28]
4. **Adjust Expected Outcomes:** Map the new degraded velocity to an xwOBA penalty. Apply this penalty mathematically to the pitcher's base probabilities in the multi-outcome Log5 calculation before generating the RNG result for the next batter.

By modeling fatigue as a continuous degradation of velocity, which subsequently modifies the Log5 base probabilities and alters the Markov chain run expectancies, the simulation achieves a fluid, statistically robust reflection of real-world baseball without resorting to hard-coded, artificial pitch limits.

True Talent vs. Variance: Stabilization Rates and BABIP

A primary architectural challenge in building a sports dynasty game is separating true talent from simulated RNG noise. Human users will invariably look at short-term data and assume an artificial change in talent. The engine must adhere to empirical stabilization rates, dictating exactly how many plate appearances (PA) or batters faced (BF) are required before the "signal" of a skill mathematically overcomes the "noise" of variance.[34] In sabermetrics, this is defined as the point at which split-half correlation reaches $R = 0.50$, allowing analysts to predict 50 percent of the future variation in a statistic.[34]

BABIP Mechanics

Batting Average on Balls in Play (BABIP) measures how often a ball put into the field of play goes for a hit, removing outcomes not affected by the opposing defense (home runs and strikeouts).[36] It is calculated as:

$$BABIP = \frac{H - HR}{AB - K - HR + SF}$$

The league average BABIP consistently hovers around.300.[37] Hitters exert significantly more control over their BABIP than pitchers do.[36] A pitcher who allows an exceptionally high percentage of hits on balls in play will almost always regress heavily to the mean over time.[37]

Empirical Stabilization Thresholds

The simulation must maintain internal, hidden "True Talent" ratings for all players, generating outcomes based on these ratings, and allowing surface statistics to stabilize naturally over the following empirically researched timelines:

Metric	Stabilization Threshold	Implication for Simulation Engine
Hitter Strikeout Rate (K%)	60 Plate Appearances (PA)	Stabilizes rapidly. Changes reflect true talent.[38]
Pitcher Strikeout Rate (K%)	70 Batters Faced (BF)	Stabilizes rapidly. Pitcher strikeout skill is highly innate.[39]
Hitter Walk Rate (BB%)	120 Plate Appearances (PA)	Stabilizes moderately fast. Reflects true plate discipline.[38]
Hitter BABIP	~800 Balls in Play (BIP)	Highly volatile. Requires nearly two full seasons to stabilize.
Pitcher BABIP	>1,000 Batters Faced (BF)	Inherently chaotic. Do not adjust pitcher ratings based on BABIP.[40]

When the engine generates a daily probability slate for an impending simulated game, it must rely exclusively on the player's long-term stabilized rating or projection. It must rigorously resist the mathematical temptation to alter a pitcher's underlying rating based on a "lucky" or "unlucky" 30-inning stretch of extreme BABIP variance.[41] The UI should present the chaotic variance to the user, forcing them to determine whether a player is experiencing a true breakout or simply statistical noise.

Player Valuation: A Simplified WAR Architecture

To facilitate roster management, trades, waiver wires, and free agency logic, the AI must

evaluate overall player value through a unified, context-neutral metric: Wins Above Replacement (WAR). A streamlined, highly efficient WAR calculation methodology is crucial for a JavaScript-based simulation engine, as recalculating thousands of complex player values per virtual day can severely throttle client-side performance.

Position Player WAR Calculation

The engine should structure Position Player WAR by summing specific run-based components to achieve Runs Above Replacement (RAR), then converting that figure to wins by dividing by the league-environment Runs Per Win (RPW) constant.[43]

$$WAR_{batter} = \frac{wRAA + BsR + Fielding + Positional + ReplacementRuns}{RPW}$$

Component Breakdown for the Engine:

1. **wRAA (Weighted Runs Above Average):** Calculates offensive contribution. $wRAA = \left(\frac{wOBA - lgwOBA}{wOBAScale}\right) \times PA$.[43] In a self-contained simulation without historical data to parse, the $wOBAScale$ constant can be safely locked to roughly 1.125 or 1.25 depending on the era settings chosen by the user.[44]
2. **Base Running Runs (BsR):** A composite of stolen base value and ground-into-double-play avoidance, measured in runs above average.[43]
3. **Fielding Runs:** The run-value of defensive plays made above or below the positional average (akin to Ultimate Zone Rating or Defensive Runs Saved).[43]
4. **Positional Adjustment:** Corrects for the defensive difficulty of the position played, placing all players on the same baseline. Scaled to a full 162-game season (or 1458 defensive innings), the standard adjustments in runs are [43]:

Position	**Adjustment (Runs/162 Games)**
Catcher (C)	+12.5
Shortstop (SS)	+7.5
Second Base (2B)	+2.5

Third Base (3B)	+2.5
Center Field (CF)	+2.5
Left Field (LF)	-7.5
Right Field (RF)	-7.5
First Base (1B)	-12.5
Designated Hitter (DH)	-17.5

5. **Replacement Level Runs:** Represents the baseline of a freely available minor-league call-up. Sabermetric consensus allocates exactly 1,000 total WAR per 2,430 MLB games, with 57% of this value (570 WAR) assigned to position players.[43] The formula is [43]:

$$\text{Replacement Runs} = \left(570 \times \frac{\text{Games Played}}{2430}\right) \times \left(\frac{RPW}{lgPA}\right) \times PA$$

6. **Runs Per Win (RPW):** Rather than complex dynamic mapping, the engine can utilize Tom Tango's simplified but highly accurate RPW formula, which matches deeper mathematical derivations within an error margin of 0.02 runs per win [43]:

$$RPW = 9 \times \left(\frac{\text{League Runs Scored}}{\text{League Innings Pitched}}\right) \times 1.5 + 3$$

Pitcher WAR Calculation

Pitcher WAR in the simulation must avoid the extreme noise of defensive variance. Therefore, it relies on Fielding Independent Pitching (FIP) rather than raw Runs Allowed (RA9).[48]

The simplified Pitcher WAR architecture for the simulation engine is:

WAR_{pitcher} = \left \times \text{LevMultiplier} + \text{Correction}

(Source: [52])

Crucial Modifiers for Pitchers:

- **Replacement Level Split:** Pitcher replacement levels vary strictly by role. Starters carry a baseline replacement level of 0.12 Wins Per Game Above Replacement (WPGAR), while pure relievers are set much lower at 0.03.[52] If a pitcher splits time in the simulation, a weighted average is used: $0.03 \times (1 - GS/G) + 0.12 \times (GS/G)$.[52]
- **Leverage Multiplier:** Reliever WAR is fundamentally distorted without adjusting for the situations they face. The FIP component must be multiplied by the average Leverage Index (pLI) at the time of the pitcher's entry into the game. As established, an average leverage is 1.0, while many elite closers will approach an average of 2.0 for the season, and mop-up relievers might sit at 0.7.[55]

Macro-Level Standings: Pythagorean Expectation Variants

As the simulation progresses through a 162-game season, the engine needs an auditing mechanism to evaluate whether a team's actual Win-Loss record aligns with its true talent, or if the team is experiencing significant variance. This is necessary for the AI to make rational trade deadline decisions rather than panicking over a mathematically unlucky record. This is achieved using Pythagorean Expectation models.[56]

Exponent Failure Modes

Bill James's original Pythagorean formula estimated a team's winning percentage ($W\%$) using an exponent of 2.0: $W\% = R^2/(R^2 + RA^2)$.[56] Empirical testing over decades of baseball later showed that an exponent of 1.83 was more accurate for average MLB run environments.[56] However, fixed-exponent models fail mechanically at mathematical extremes. If users tweak the simulation settings to generate a highly extreme run environment (e.g., recreating the Dead-ball era or an extremely high-octane offensive league), a fixed 1.83 exponent breaks down entirely.

The Pythagenpat Implementation

The most accurate, context-dependent variant of Pythagorean expectation for modern software simulations is the Pythagenpat formula, developed by David Smyth.[59] Pythagenpat dynamically derives the exponent directly from the total run environment of the league or the specific team, yielding higher exponents in higher-scoring environments.[59]

The Pythagenpat exponent (X) is defined mathematically as [60]:

$$X = \left(\frac{R_{scored} + R_{allowed}}{Games} \right)^{0.285}$$

Or, using Runs Per Game (RPG) [60]:

$$X = RPG^{0.285}$$

The Expected Winning Percentage is then calculated using that specific exponent:

$$W\% = \frac{R^X}{R^X + RA^X}$$

The fundamental mathematical justification for Pythagenpat over older formulas (like Pythagenport) is its adherence to the "(1,1) point" theorem.[60] At the absolute minimum possible run environment of exactly 1 Run Per Game, the relationship between runs and wins must be perfectly linear (a team wins every time they score and loses every time they allow a run). For the Pythagorean formula to hold true at 1 RPG, the exponent X must equal exactly 1.[60] Earlier dynamic models produced an exponent of 0.45 at 1 RPG, invalidating them at the mathematical extremes.[60] Because $1^{0.285} = 1$, Pythagenpat perfectly maintains structural integrity across any theoretical run environment the simulation engine might produce.[60]

In the simulation engine, the derivative of the Run Differential with respect to Pythagenpat Winning Percentage can be used to calculate Marginal Runs Per Win (RPW)—the exact number of runs necessary to add one more win to a team's record in a given context [60]:

$$RPW = \frac{2 \cdot RPG \cdot (RR^X + 1)^2 \cdot \left(0.5 - \frac{RD}{2 \cdot RPG}\right)^2}{X \cdot RR^{X-1}}$$

Where RR is the Run Ratio (R/RA) and RD is the Run Differential per game.[60] For an average team in the simulation, this simplifies drastically to a slope of $X/(2 \cdot RPG)$.[60]

Temporal Dynamics: Aging Curves and Career Trajectories

In a dynasty format, player development and physical degradation are the central engines that drive the multi-season economy. Players do not age linearly. Instead, they follow an aging curve that reflects skill acquisition, physical peak, and eventual biomechanical decline. Sabermetric aging curves are predominantly built using the "Delta Method," which pairs player seasons to find the average year-over-year difference (the delta) in a specific statistic, chaining these pairs together to form a lifetime mathematical trajectory.[61] The React state must apply these

deltas iteratively every simulated off-season.

General Aging and Decline Rates

The overarching consensus in sabermetrics suggests that position players generally peak around ages 26 to 28.[63] Base running speed and defensive range tend to peak earliest (early 20s), while plate discipline and power metrics (ISO, BB%) can peak slightly later.[63] Once a player crests the age of 30, the standard baseline rule of thumb for the simulation is an expected loss of approximately 0.5 WAR per year of value due strictly to physical aging.[63]

Positional Nuances

The simulation cannot apply a universal, flat aging curve to all positions. Defensive demands and the specific biomechanics of each position alter the trajectory heavily:

- **Shortstops and Center Fielders:** Defensive metrics for shortstops show a physical peak between the ages of 22 and 24. After this point, the average matched-pair decline is roughly nine plays per year (equating to a loss of seven defensive runs annually).[66] While their offensive peak may occur later, their defensive peak is exceptionally early, requiring the simulation to degrade their range ratings well before their hitting ratings.
- **Third Basemen:** Third basemen possess a uniquely late offensive peak, largely due to their reliance on secondary averages (power and walk rate) rather than pure foot speed. They are one of the few positions where average value at age 29 is empirically higher than at age 25.[65] However, they suffer a rapid decline ("cliff") around age 35 when fast-twitch reflexes diminish.[65]
- **Catchers:** The physical toll of the catcher position has historically led to assumptions of early decline. However, empirical delta-method tracking reveals that catchers' offensive profiles peak from ages 25 to 28 and follow a decline trajectory nearly in lock-step with other position players until age 32.[67] Post-32, the surviving catchers actually appear to age *better* than average position players, though this is heavily influenced by survivorship bias (only the elite hitting catchers are retained or moved to First Base/DH).[67] The simulation should degrade catcher defensive metrics linearly, but leave their bat intact until age 32.
- **Pitchers:** Pitchers experience the most immediate physical decline. Fastball velocity peaks very early in a career (around age 25) and then begins a slow, tapered decline through the late 20s, accelerating drastically into the 30s.[63] Aging for pitchers in the engine should not be applied directly to their strikeout or walk ratings; instead, aging must be tied directly to a decay function on velocity, which subsequently degrades their strikeout rates and contact suppression through the physics formulas outlined in the fatigue section.

Synthesizing the Simulation Engine Architecture

The construction of a self-contained, browser-based React/JSX simulation engine demands that complex statistical realities be distilled into highly efficient mathematical procedures. By utilizing the framework provided in this report, an engine can accurately simulate the sport

from the micro to the macro level without requiring a server to crunch the numbers.

The process flows sequentially during a simulated game: The AI evaluates the **Leverage Index** using Tom Tango's ΔWE estimation to select the optimal pitcher and batter.[21] The **Multi-Outcome Log5** (corrected with Bayesian hierarchical priors and ground-ball interaction modifiers, and adjusted dynamically for the pitcher's **Continuous Velocity Degradation**) calculates the exact probability distribution for the plate appearance.[2] The RNG resolves the outcome, updating the **Markov Chain** state vector to manage base running and expected runs.[13]

Over the course of 162 games, this loop accrues statistics that accurately inform the simplified **Positional WAR** and pitcher FIP calculations.[43] Simultaneously, the league-wide **Pythagenpat** formula validates that team win-loss records align perfectly with the fundamental mathematics of the simulated run environment.[60] Finally, year-over-year, the **Delta Method Aging Curves** systematically advance or erode player skill levels based on positional specifics.[61] By isolating these mathematical truths from statistical noise, the resulting application will be a dynamic, deeply realistic, and perfectly self-contained dynasty ecosystem.

Works cited

1. Probabilities of Victory in Head-to-Head Team Matchups - SABR.org, accessed February 24, 2026, https://sabr.org/journal/article/probabilities-of-victory-in-head-to-head-team-matchups/
2. Matchup Probabilities in Major League Baseball – Society for ..., accessed February 24, 2026, https://sabr.org/journal/article/matchup-probabilities-in-major-league-baseball/
3. Why xwOBA (expected weighted on-base average) is useful | Sox On 35th, accessed February 24, 2026, https://www.soxon35th.com/why-xwoba-expected-weighted-on-base-average-is-useful/amp/
4. How specific should you get for an optimal log5 probability slate? : r/Sabermetrics - Reddit, accessed February 24, 2026, https://www.reddit.com/r/Sabermetrics/comments/i9kp3e/how_specific_should_you_get_for_an_optimal_log5/
5. Sabermetric Research: August 2016 - Phil Birnbaum, accessed February 24, 2026, http://blog.philbirnbaum.com/2016/08/
6. Modeling the probability of a batter/pitcher matchup event: A Bayesian approach - PMC, accessed February 24, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC6192592/
7. Modeling the Probability of a Strikeout for a Batter/Pitcher Matchup - IEEE Computer Society, accessed February 24, 2026, https://www.computer.org/csdl/journal/tk/2015/09/07069266/13rRUyp7tXj
8. Modeling the Probability of a Strikeout for a Batter/Pitcher Matchup - IEEE Xplore,

accessed February 24, 2026, https://ieeexplore.ieee.org/iel7/69/7177178/07069266.pdf
9. Strat-O-Matic Platoon Splits - Sports Mogul Fileserver, accessed February 24, 2026, https://files.sportsmogul.com/articles/strat-platoon-splits.pdf
10. Checking the Numbers: Contextual Platooning | Baseball Prospectus, accessed February 24, 2026, https://www.baseballprospectus.com/article.php?articleid=9167
11. Estimating Hitter Platoon Skill | FanGraphs Baseball, accessed February 24, 2026, https://blogs.fangraphs.com/estimating-hitter-platoon-skill/
12. Raising Aces: Splitting the Platoon: Lefty-philic Pitchers | Baseball Prospectus, accessed February 24, 2026, https://www.baseballprospectus.com/news/article/20784/raising-aces-splitting-the-platoon-lefty-philic-pitchers/
13. A Markov Chain Approach to Baseball - NJIT, accessed February 24, 2026, https://web.njit.edu/~bukiet/Papers/ball.pdf
14. An Intuitive Markov Chain Lesson From Baseball - PubsOnLine, accessed February 24, 2026, https://pubsonline.informs.org/doi/pdf/10.1287/ited.5.1.47
15. The Markov Chain Model of Baseball - statshacker, accessed February 24, 2026, http://statshacker.com/blog/2018/05/07/the-markov-chain-model-of-baseball/
16. RE24 | Sabermetrics Library, accessed February 24, 2026, https://library.fangraphs.com/misc/re24/
17. wSLG: Redefining the Relative Values of Various Hits in Baseball - UNC Press | Janeway, accessed February 24, 2026, https://janeway.uncpress.org/ms/article/1266/galley/2180/view/
18. A Markov Chain Model for Predicting Major League Baseball - The University of Texas at Austin, accessed February 24, 2026, https://repositories.lib.utexas.edu/bitstreams/88eb5af3-a824-487c-829e-46def421a869/download
19. analysis of a baseball simulation game using markov chains - Lehigh University, accessed February 24, 2026, https://www.lehigh.edu/~dmd1/apba3.pdf
20. The Run Expectancy Matrix, Reloaded for the 2020s - FanGraphs Baseball, accessed February 24, 2026, https://blogs.fangraphs.com/the-run-expectancy-matrix-reloaded-for-the-2020s/
21. Leverage Index (LI) | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/advanced-stats/leverage-index
22. LI | Sabermetrics Library - FanGraphs Library, accessed February 24, 2026, https://library.fangraphs.com/misc/li/
23. UConn Baseball Reliever Lane Optimization Tool - Digital Commons @ UConn - University of Connecticut, accessed February 24, 2026, https://digitalcommons.lib.uconn.edu/cgi/viewcontent.cgi?article=2044&context=srhonors_theses
24. Get to Know: Leverage Index - FanGraphs Baseball, accessed February 24, 2026, https://blogs.fangraphs.com/get-to-know-leverage-index/
25. Calculating leverage Index for individual plate appearances : r/Sabermetrics -

Reddit, accessed February 24, 2026, https://www.reddit.com/r/Sabermetrics/comments/1d89vu6/calculating_leverage_index_for_individual_plate/
26. Rethinking Acute to Chronic Workload for Pitchers - ArmCare, accessed February 24, 2026, https://blog.armcare.com/rethinking-acute-to-chronic-workload-for-pitchers/
27. An Introduction to Fatigue Units: A New Method for Evaluating Workloads, accessed February 24, 2026, https://fantasy.fangraphs.com/an-introduction-to-fatigue-units-a-new-method-for-evaluating-workloads/
28. Relationship Between Pitcher Fatigue and Medial Elbow Torque in Baseball Pitchers: A Simulated Game Analysis - PMC, accessed February 24, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC6083763/
29. What is the state of the art on understanding pitcher fatigue? : r/baseball - Reddit, accessed February 24, 2026, https://www.reddit.com/r/baseball/comments/1p2r0ig/what_is_the_state_of_the_art_on_understanding/
30. Expected Weighted On-base Average (xwOBA) | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/statcast/expected-woba
31. Statcast Expected wOBA, xBA, xSLG | baseballsavant.com - MLB.com, accessed February 24, 2026, https://baseballsavant.mlb.com/leaderboard/expected_statistics
32. An Introduction to Expected Weighted On-Base Average (xwOBA) - MLB Technology Blog, accessed February 24, 2026, https://technology.mlblogs.com/an-introduction-to-expected-weighted-on-base-average-xwoba-29d6070ba52b
33. The Real Influence of Velocity - Pitcher List, accessed February 24, 2026, https://pitcherlist.com/the-real-influence-of-velocity/
34. Resident Fantasy Genius: When Pitchers' Stats Stabilize | Baseball Prospectus, accessed February 24, 2026, https://www.baseballprospectus.com/fantasy/article/14293/resident-fantasy-genius-when-pitchers-stats-stabilize/
35. Resident Fantasy Genius: When Hitters' Stats Stabilize - Baseball Prospectus, accessed February 24, 2026, https://www.baseballprospectus.com/fantasy/article/14215/resident-fantasy-genius-when-hitters-stats-stabilize/
36. BABIP | Sabermetrics Library, accessed February 24, 2026, https://library.fangraphs.com/pitching/babip/
37. Batting Average on Balls in Play (BABIP) | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/advanced-stats/babip
38. Using Plate Discipline Stats in Fantasy Baseball | FantasyPros, accessed February 24, 2026, https://www.fantasypros.com/2017/02/using-plate-discipline-stats-in-fantasy-baseball/
39. Baseball Therapy: Should I Worry About My Favorite Pitcher?, accessed February

24, 2026, https://www.baseballprospectus.com/news/article/20516/baseball-therapy-should-i-worry-about-my-favorite-pitcher/
40. Panic Or Chill? - by Michael Cecchini - Michael's Substack, accessed February 24, 2026, https://michaelcecchini.substack.com/p/panic-or-chill
41. Understanding BABIP Pt. 1: Pitchers, accessed February 24, 2026, https://pitcherlist.com/understanding-babip-pt-1-pitchers/
42. Carleton: Rethinking randomness among pitchers and their BABIPs - SABR.org, accessed February 24, 2026, https://sabr.org/latest/carleton-rethinking-randomness-among-pitchers-and-their-babips/
43. WAR for Position Players | Sabermetrics Library - FanGraphs Library, accessed February 24, 2026, https://library.fangraphs.com/war/war-position-players/
44. WAR(for position players), and how to calculate it. Both a "Simple" and Full Version - Reddit, accessed February 24, 2026, https://www.reddit.com/r/SuperMegaBaseball/comments/14dxjm7/warfor_position_players_and_how_to_calculate_it/
45. WAR calculation - OOTP Developments Forums, accessed February 24, 2026, https://forums.ootpdevelopments.com/showthread.php?t=231816
46. Major League Leaderboards - 2025 - Batting | FanGraphs Baseball, accessed February 24, 2026, https://www.fangraphs.com/leaders/major-league
47. Advanced Stats | Glossary - MLB.com, accessed February 24, 2026, https://www.mlb.com/glossary/advanced-stats
48. Sabermetrics 101: Understanding the Calculation of WAR - Samford University, accessed February 24, 2026, https://www.samford.edu/sports-analytics/fans/2023/Sabermetrics-101-Understanding-the-Calculation-of-WAR
49. Positional Adjustment | Sabermetrics Library, accessed February 24, 2026, https://library.fangraphs.com/misc/war/positional-adjustment/
50. Calculating the WAR Statistic : 5 Steps (with Pictures) - Instructables, accessed February 24, 2026, https://www.instructables.com/Calculating-the-WAR-Statistic/
51. What is WAR in Baseball? An Introduction to College WAR, accessed February 24, 2026, https://www.drivelinebaseball.com/2020/06/an-introduction-to-cwar/
52. Calculating Pitcher WAR, A Complete Example | Sabermetrics Library, accessed February 24, 2026, https://library.fangraphs.com/calculating-pitcher-war-a-complete-example/
53. WAR for Pitchers - FanGraphs Library, accessed February 24, 2026, https://library.fangraphs.com/war/calculating-war-pitchers/
54. Calculating Pitching WAR : r/Sabermetrics - Reddit, accessed February 24, 2026, https://www.reddit.com/r/Sabermetrics/comments/18y60cw/calculating_pitching_war/
55. Pitcher WAR Calculations and Details - Baseball-Reference.com, accessed February 24, 2026, https://www.baseball-reference.com/about/war_explained_pitch.shtml
56. Pythagorean Expectation in Baseball - AWS, accessed February 24, 2026,

http://rstudio-pubs-static.s3.amazonaws.com/22647_ef62be908e254269ba30ad24c1cf8ad4.html
57. Pythagorean Win-Loss | Sabermetrics Library, accessed February 24, 2026, https://library.fangraphs.com/principles/expected-wins-and-losses/
58. Pythagorean Theorem of Baseball - BR Bullpen, accessed February 24, 2026, https://www.baseball-reference.com/bullpen/Pythagorean_Theorem_of_Baseball
59. Pythagorean expectation - Wikipedia, accessed February 24, 2026, https://en.wikipedia.org/wiki/Pythagorean_expectation
60. View details for Pythagenpat - Baseball Prospectus | Glossary, accessed February 24, 2026, https://legacy.baseballprospectus.com/glossary/index.php?mode=viewstat&stat=136
61. The Delta Method, Revisited: Rethinking Aging Curves | Baseball Prospectus, accessed February 24, 2026, https://www.baseballprospectus.com/news/article/59972/the-delta-method-revisited/
62. How Do Catchers Age? – Investigating the Lifestyle of a Backstop, accessed February 24, 2026, https://rmsabr.org/blog/how-do-catchers-age-investigating-the-lifestyle-of-a-backstop/
63. Aging Curve | Sabermetrics Library, accessed February 24, 2026, https://library.fangraphs.com/principles/aging-curve/
64. How power ages (It might surprise you) - Driveline Baseball, accessed February 24, 2026, https://www.drivelinebaseball.com/2025/10/how-power-ages-it-might-surprise-you/
65. Lies, Damned Lies: A New Look at Aging - Baseball Prospectus, accessed February 24, 2026, https://www.baseballprospectus.com/news/article/4464/lies-damned-lies-a-new-look-at-aging/
66. Fielding aging curves | The Hardball Times - FanGraphs, accessed February 24, 2026, https://tht.fangraphs.com/fielding-aging-curves/
67. Catcher Aging is a Curve, Not a Cliff | FanGraphs Baseball, accessed February 24, 2026, https://blogs.fangraphs.com/catcher-aging-is-a-curve-not-a-cliff/
68. Fangraphs: Catcher Aging Curve - MLB - Orioles Hangout Community, accessed February 24, 2026, https://forum.orioleshangout.com/topic/16485-fangraphs-catcher-aging-curve/